



Firestore Auth y rutas protegidas en Vue 3

Identidad, control de sesión y seguridad de datos
con VueFire, Composition API y Vue Router.

EL CONTEXTO: DE UNA APP ABIERTA A UN ENTORNO SEGURO

 **AYER**

Ayer: Interfaz pública y base de datos sin restricciones.



 **HOY**

Hoy: Identidad validada, experiencia condicional (UX) y datos protegidos en el backend.



¿Qué es Firebase Authentication?

Un servicio que simplifica el registro, inicio de sesión y gestión de sesión de usuarios.



Identidad

Registrar y
validar
usuarios.



Sesión

Iniciar y
cerrar sesión
de forma
segura.



Seguridad

Mantener la
sesión activa
mediante tokens
automáticos,
sin gestión
manual.

Configuración Inicial

Terminal

```
pnpm add firebase vuefire vue-router
```

firebase.js

```
import { initializeApp } from 'firebase/app'
import { getFirestore } from 'firebase/firestore'
import { getAuth } from 'firebase/auth'

export const firebaseApp = initializeApp(firebaseConfig)
export const db = getFirestore(firebaseApp)
export const auth = getAuth(firebaseApp)
```

Activando VueFire Auth

"main.js" or "main.ts"

```
import { VueFire, VueFireAuth } from 'vuefire'

createApp(App)
  .use(VueFire, {
    firebaseApp,
    modules: [VueFireAuth()]
  })
  .use(router)
  .mount('#app')
```

Inyecta el módulo de autenticación en la cadena de plugins, conectando Firebase Auth con el ecosistema reactivo de Vue.

Lógica de Registro de Usuarios

Code Editor

```
<script setup>
import { ref } from 'vue'
import { createUserWithEmailAndPassword } from 'firebase/auth'
import { auth } from '@/firebase'

const email = ref('')
const password = ref('')
const error = ref(null)

async function registerUser() {
  try {
    await createUserWithEmailAndPassword(auth, email.value,
      password.value)
  } catch (err) {
    error.value = err.message
  }
}
</script>
```

Input de Usuario
(Refs reactivas)



Firebase API
(createUserWith...)



Estado
(Éxito o Error.value)

Interfaz del Formulario de Registro

Code Editor

```
<template>
  <form @submit.prevent="register">
    <input v-model="email" type="email"
      placeholder="Correo" />
    <input v-model="password" type="password"
      placeholder="Contraseña" />
    <button type="submit">Crear cuenta</button>
    <p v-if="error" class="error-msg">{{ error }}</p>
  </form>
</template>
```

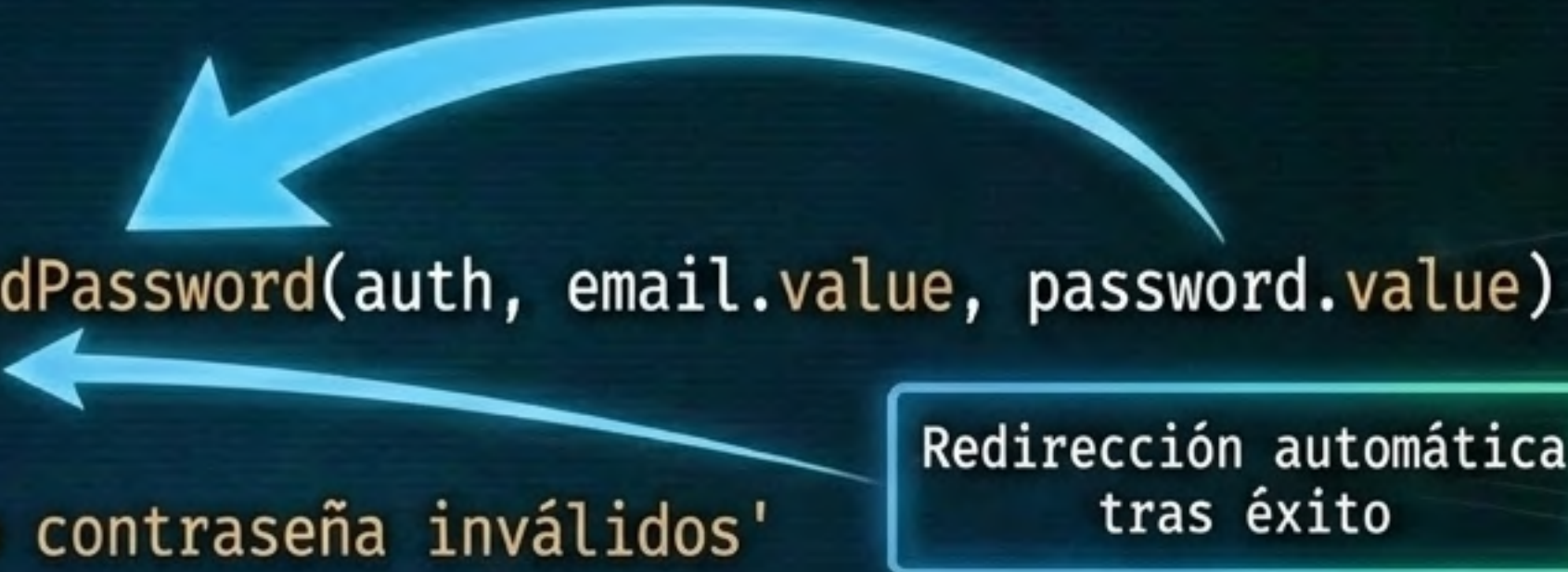
Crear cuenta

El correo ya está en uso

Inicio de Sesión y Redirección

Code Editor

```
1 import { signInWithEmailAndPassword } from 'firebase/auth'
2 import { useRouter } from 'vue-router'
3
4 const router = useRouter()
5
6 async function login() {
7   try {
8     await signInWithEmailAndPassword(auth, email.value, password.value)
9     router.push('/michis')
10   } catch (err) {
11     error.value = 'Correo o contraseña inválidos'
12   }
13 }
```



A blue curved arrow originates from the `router.push('/michis')` line (line 9) and points to the `login()` function definition (line 6). A second blue curved arrow originates from the `error.value = 'Correo o contraseña inválidos'` line (line 11) and points to the same `login()` function definition. A text box on the right contains the text "Redirección automática tras éxito".

Redirección automática
tras éxito

Usuario Actual y Cierre de Sesión

`useCurrentUser()` entrega una referencia reactiva al usuario autenticado actual, actualizando la UI automáticamente en toda la app.



Code Editor

```
const user = useCurrentUser()
async function logout() {
  await signOut(auth)
}
```

```
<p v-if="user">Sesión: {{ user.email }}</p>
<button v-if="user" @click="logout">Cerrar sesión</button>
```

Definiendo Rutas Privadas (meta)



Code Editor

```
1 const routes = [  
2   { path: '/login', component: LoginView },  
3   { path: '/register', component: RegisterView },  
4   { path: '/register', component: RegisterView },  
5   {  
6     path: '/michis',  
7     component: MichisView,  
9     meta: { requiresAuth: true }  
10  }  
11 ]
```

Navigation Guard: El Guardia de Seguridad

Code Editor

```
import { getCurrentUser } from 'vuefire'

router.beforeEach(async (to) => {
  const user = await getCurrentUser()

  if (to.meta.requiresAuth && !user) {
    return '/login'
  }
})
```



Navegación Condicional (UX)

Code Editor

```
<nav>
  <RouterLink to="/">Inicio</RouterLink>

  <!-- Sólo visitantes -->
  <RouterLink v-if="!user" to="/login">Login</RouterLink>
  <RouterLink v-if="!user" to="/register">Registro</RouterLink>

  <!-- Sólo autenticados -->
  <RouterLink v-if="user" to="/michis">Michis</RouterLink>
  <button v-if="user" @click="logout">Salir</button>
</nav>

<RouterView />
```

[Visitante]

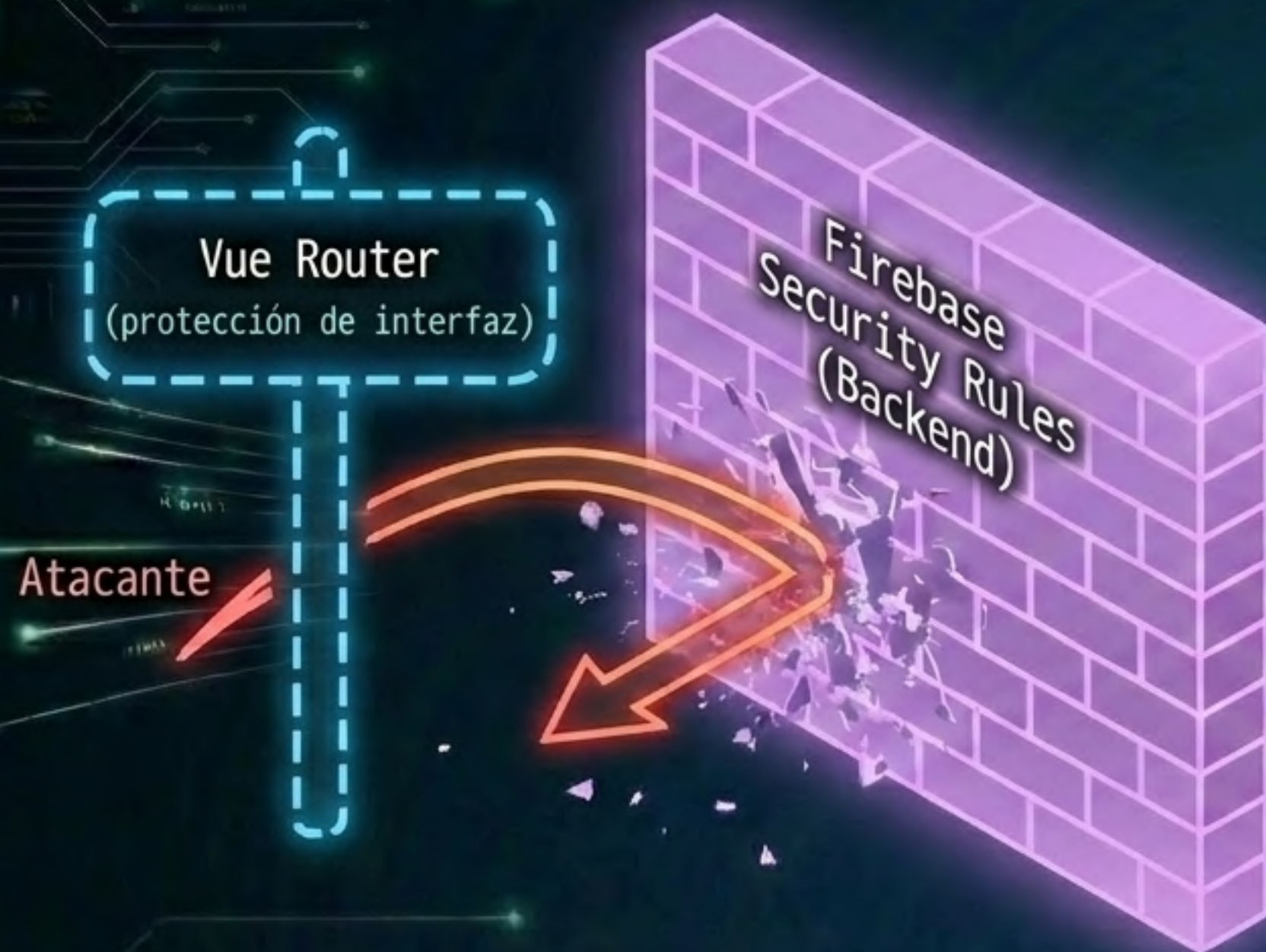
Inicio Login Registro

[Autenticado]

Inicio Michis **Salir**

La Verdadera Seguridad: Firestore Rules

Proteger una ruta en Vue solo oculta la interfaz gráfica.
La seguridad real siempre reside en el servidor.



Code Editor

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    match /michis/{michiId} {
      allow read, write: if request.auth != null;
    }
  }
}
```

Auth + Firestore: Uniendo las piezas

Ahora que las Reglas exigen autenticación, podemos vincular cada documento a su creador.

Code Editor

```
const user = currentUser()

async function addMichi(name) {
  if (!user.value) return

  await addDoc(collection(db, 'michis'), {
    nombre: name,
    ownerId: user.value.uid
  })
}
```



Pro-Tip: Rastro de auditoría y propiedad de los datos.

Resumen de Arquitectura de Seguridad



Auth SDK

Crea y valida credenciales contra Google.



useCurrentUser()

Entrega el estado reactivo de la sesión para componentes Vue.



Vue Router

protege la navegación y la experiencia de usuario.



Security Rules

protegen los datos reales en Firebase.